L2L Framework: 5-Fold Cross-Validation, CFA, IRT, and Demographic Profiling

Script 02 — Psychometric Infrastructure for Item Imputation

Kevin Linares

2026-03-11

Overview

This script builds the complete psychometric infrastructure for the L2L item imputation experiment. All downstream scripts load the outputs produced here to ensure every method is evaluated on identical held-out cases with identical psychometric profiles.

Four analysis components are fitted per fold:

1. **CFA:** Unidimensional trust factor (ML estimation). Provides factor scores, loadings, fit indices, and composite reliability.
2. **GRM/IRT:** Unidimensional Graded Response Model with SEs. Provides discrimination parameters, Q3 diagnostics, theta estimates.
3. **Demographic Profiling:** For each demographic variable, computes
   (a) eta-squared with theta to screen which demographics carry meaningful signal, (b) per-item group means for direction and magnitude, (c) model-based conditional probabilities $P(X_j \geq k \mid \bar{\theta}_{\text{group}})$ for prompt context, and (d) conditional response distributions $P(X_j = k \mid \text{group})$ for the strongest associations.
4. **Masking Table:** Random assignment of 1–4 masked items per test respondent, shared across all downstream scripts.

Changes from Previous Version

- **Dropped:** Measurement invariance (MI) and DIF (lordif). The diagnostic analysis showed DIF R^2 values below 0.03 across all items, meaning DIF flags added prompt tokens without actionable signal. The lordif dependency is removed.
- **Dropped:** Theta regression. Replaced by demographic profiling which produces the same information (demographic × latent relationship) in a format directly usable by LLM prompts: model-based probabilities rather than regression coefficients.
- **Added:** Employment as the fifth demographic variable, resolving the mismatch between Scripts 03 and 04.

- **Added:** Demographic screening via eta-squared threshold, computed per survey independently.
- **Added:** Model-based probability context per demographic group, transforming IRT parameters into $P(X_j \geq k \mid \theta)$ that LLMs can interpret without psychometric expertise.

Key Design Decisions

- **5-fold CV** (not 10) for larger training folds (~80% of N), required for stable CFA estimation.
- **IRT reverse-coding for LB** so higher theta = higher trust.
- **Demographic screening threshold:** eta² 0.005 with theta. Demographics below this threshold are excluded from prompts for that survey to avoid adding noise.
- **Same five demographics tested** for both surveys; survivors determined independently per survey.

Setup

```
library(rsample)
library(lavaan)
library(semTools)
library(mirt)
library(ggridges)
library(viridis)
library(tidyverse)

set.seed(721)

results_dir <- "D:/repos/UMD_classes_code/TSE_II_SURV721/results"
survey_list <- readRDS(file.path(results_dir, "survey_data.rds"))
```

Survey Configuration

```
config <- list(
  lapop = list(
    data            = survey_list$lapop,
    demo_vars       = c("male", "age_cat", "Edu", "Urban",
                        "Employment"),
    trust_items     = c(
      "justice_system", "electoral_tribunal", "armed_forces",
      "legislature", "public_ministry", "police", "auditor",
      "political_parties", "supreme_court", "municipality",
      "media", "elections"
    ),
```

```
    cfa_syntax = '
      Trust =~ justice_system + electoral_tribunal + armed_forces +
               legislature + public_ministry + police + auditor +
               political_parties + supreme_court + municipality +
               media + elections
      armed_forces ~~ legislature
      public_ministry ~~ auditor
    ',
    factor_names    = "Trust",
    trust_ascending = TRUE
  ),
  lb = list(
    data            = survey_list$lb,
    demo_vars       = c("male", "age_cat", "Edu", "Urban",
                        "Employment"),
    trust_items     = c(
      "Congress", "Natl_Govt", "Judiciary", "Parties",
      "Electoral_Inst", "President"
    ),
    cfa_syntax = '
      Political =~ Congress + Natl_Govt + Judiciary + Parties +
                   Electoral_Inst + President
      Natl_Govt ~~ President
    ',
    factor_names    = "Political",
    trust_ascending = FALSE
  )
)

# Demographic screening threshold
ETA_SQ_THRESHOLD <- 0.005

# Q3 threshold for local dependence
Q3_THRESHOLD <- 0.10

# Number of folds
N_FOLDS <- 5L
```

Helper Functions

```
# Label trust level relative to training distribution
label_trust <- function(fs_test, fs_train) {
  m <- mean(fs_train)
  s <- sd(fs_train)
  case_when(
```

```
    fs_test < (m - 1.0 * s) ~ "VERY LOW",
    fs_test < (m - 0.5 * s) ~ "LOW",
    fs_test < (m + 0.5 * s) ~ "MODERATE",
    fs_test < (m + 1.0 * s) ~ "HIGH",
    TRUE                    ~ "VERY HIGH"
  )
}

# Verbal labels for observed responses
lapop_trust_labels <- c(
  "1" = "No trust at all", "2" = "Very little trust",
  "3" = "Little trust",    "4" = "Some trust",
  "5" = "Moderate trust",  "6" = "High trust",
  "7" = "A lot of trust"
)

lb_trust_labels <- c(
  "1" = "A lot of trust", "2" = "Some trust",
  "3" = "Little trust",   "4" = "No trust"
)

# Eta-squared from one-way ANOVA
compute_eta_sq <- function(y, x) {
  valid <- !is.na(y) & !is.na(x)
  y <- y[valid]; x <- x[valid]
  if (length(unique(x)) < 2 || length(y) < 10) {
    return(tibble(eta_sq = NA_real_, F_stat = NA_real_,
                  p_value = NA_real_, n = length(y)))
  }
  fit <- aov(y ~ x)
  ss  <- summary(fit)[[1]]
  tibble(
    eta_sq  = round(ss[["Sum Sq"]][1] / sum(ss[["Sum Sq"]]), 4),
    F_stat  = round(ss[["F value"]][1], 2),
    p_value = round(ss[["Pr(>F)"]][1], 4),
    n       = length(y)
  )
}
```

Build 5-Fold CV Splits

```
build_survey_folds <- function(cfg, survey_name) {
  full_data <- cfg$data |>
    mutate(row_id = row_number())
```

```r
  cv_obj <- vfold_cv(full_data, v = N_FOLDS)

  map_dfr(seq_len(nrow(cv_obj)), function(fold_i) {
    sp <- cv_obj$splits[[fold_i]]
    bind_rows(
      training(sp) |> mutate(split = "train"),
      testing(sp)  |> mutate(split = "test")
    ) |>
      mutate(survey = survey_name, fold_id = fold_i) |>
      select(survey, fold_id, split, row_id, everything())
  })
}

folds_list <- list(
  lapop = build_survey_folds(config$lapop, "lapop"),
  lb    = build_survey_folds(config$lb,    "lb")
)

walk(names(folds_list), function(s) {
  cat(s, ":", nrow(folds_list[[s]]), "rows x",
      ncol(folds_list[[s]]), "columns\n")
  folds_list[[s]] |>
    count(fold_id, split) |>
    pivot_wider(names_from = split, values_from = n) |>
    print()
  cat("\n")
})
```

```
lapop : 7150 rows x 23 columns
# A tibble: 5 x 3
  fold_id  test train
    <int> <int> <int>
1       1   286  1144
2       2   286  1144
3       3   286  1144
4       4   286  1144
5       5   286  1144

lb : 5365 rows x 17 columns
# A tibble: 5 x 3
  fold_id  test train
    <int> <int> <int>
1       1   215   858
2       2   215   858
3       3   215   858
4       4   214   859
5       5   214   859
```

Component 1: Per-Fold CFA

```
fit_fold_cfa <- function(train_df, test_df, cfg, survey, fold_id) {

  factor_name  <- cfg$factor_names
  label_lookup <- if (factor_name == "Trust") {
    lapop_trust_labels
  } else {
    lb_trust_labels
  }
  trust_items <- cfg$trust_items

  trust_train <- train_df |>
    select(all_of(trust_items)) |>
    mutate(across(everything(), as.numeric))
  trust_test <- test_df |>
    select(all_of(trust_items)) |>
    mutate(across(everything(), as.numeric))

  item_min <- min(unlist(trust_train), na.rm = TRUE)
  item_max <- max(unlist(trust_train), na.rm = TRUE)

  cfa_fit <- cfa(cfg$cfa_syntax, data = trust_train,
                 estimator = "ML", meanstructure = TRUE)

  params_df <- parameterEstimates(cfa_fit)
  load_df   <- params_df |> filter(op == "=~")
  intcpt_df <- params_df |> filter(op == "~1",
                                   lhs %in% trust_items)

  fs_train <- as.numeric(
    lavPredict(cfa_fit, newdata = trust_train,
               method = "regression"))
  fs_test <- as.numeric(
    lavPredict(cfa_fit, newdata = trust_test,
               method = "regression"))

  ecdf_fn      <- ecdf(fs_train)
  pctile_train <- round(ecdf_fn(fs_train) * 100)
  pctile_test  <- round(ecdf_fn(fs_test)  * 100)
  label_train  <- label_trust(fs_train, fs_train)
  label_test   <- label_trust(fs_test,  fs_train)

  # Expected item responses from CFA
  compute_expected <- function(fs_vec) {
    exp_cols <- map(trust_items, function(item) {
      ld  <- load_df$est[load_df$rhs == item]
```

```
    int <- intcpt_df$est[intcpt_df$lhs == item]
    stopifnot(length(ld) == 1, length(int) == 1)
    raw_val   <- int + ld * fs_vec
    truncated <- pmax(item_min, pmin(item_max, raw_val))
    rounded   <- as.integer(round(truncated))
    verbal    <- unname(label_lookup[as.character(rounded)])
    tibble(!!paste0("exp_", item) := verbal)
  })
  list_cbind(exp_cols)
}

exp_train <- compute_expected(fs_train)
exp_test  <- compute_expected(fs_test)

fs_col     <- paste0("fs_",     factor_name)
pctile_col <- paste0("pctile_", factor_name)
label_col  <- paste0("label_",  factor_name)

cfa_tbl <- bind_rows(
  tibble(survey = survey, fold_id = fold_id, split = "train",
         row_id = train_df$row_id,
         !!fs_col := fs_train, !!pctile_col := pctile_train,
         !!label_col := label_train) |> bind_cols(exp_train),
  tibble(survey = survey, fold_id = fold_id, split = "test",
         row_id = test_df$row_id,
         !!fs_col := fs_test, !!pctile_col := pctile_test,
         !!label_col := label_test) |> bind_cols(exp_test)
)

# CFA parameters (long format)
std_sol <- standardizedSolution(cfa_fit) |>
  as_tibble() |>
  select(op, lhs, rhs, std.all = est.std)

cfa_params <- parameterEstimates(cfa_fit) |>
  as_tibble() |>
  left_join(std_sol, by = c("op", "lhs", "rhs")) |>
  filter(op %in% c("=~", "~1")) |>
  mutate(survey = survey, fold_id = fold_id) |>
  select(survey, fold_id, op, lhs, rhs, est, se, z,
         pvalue, std.all)

# Fit indices
fit_vals <- fitMeasures(cfa_fit,
                        c("cfi", "tli", "rmsea", "srmr")) |>
  as.list() |> as_tibble()

omega_val  <- tryCatch(as.numeric(compRelSEM(cfa_fit)),
```

```
                          error = function(e) NA_real_)
ave_val    <- tryCatch(as.numeric(AVE(cfa_fit)),
                          error = function(e) NA_real_)
factor_var <- lavInspect(cfa_fit, "est")$psi[1, 1]

cfa_fit_row <- fit_vals |>
  mutate(survey = survey, fold_id = fold_id,
         split = "train",
         omega = omega_val, ave = ave_val,
         factor_var = factor_var) |>
  select(survey, fold_id, split, cfi, tli, rmsea, srmr,
         omega, ave, factor_var)

# Test-fold CFA baseline (H2: ground truth for imputation
# vs. deletion comparison). Refit CFA on test data to get
# complete-data measurement properties for comparison.
cfa_fit_test <- tryCatch({
  test_cfa <- cfa(cfg$cfa_syntax, data = trust_test,
                  estimator = "ML", meanstructure = TRUE)
  test_fit_vals <- fitMeasures(test_cfa,
                               c("cfi", "tli", "rmsea",
                                 "srmr")) |>
    as.list() |> as_tibble()
  test_omega <- tryCatch(as.numeric(compRelSEM(test_cfa)),
                         error = function(e) NA_real_)
  test_ave   <- tryCatch(as.numeric(AVE(test_cfa)),
                         error = function(e) NA_real_)
  test_fvar  <- lavInspect(test_cfa, "est")$psi[1, 1]

  test_fit_vals |>
    mutate(survey = survey, fold_id = fold_id,
           split = "test",
           omega = test_omega, ave = test_ave,
           factor_var = test_fvar) |>
    select(survey, fold_id, split, cfi, tli, rmsea, srmr,
           omega, ave, factor_var)
}, error = function(e) {
  cat(sprintf("    Test-fold CFA failed: %s\n", e$message))
  tibble(survey = survey, fold_id = fold_id,
         split = "test",
         cfi = NA_real_, tli = NA_real_,
         rmsea = NA_real_, srmr = NA_real_,
         omega = NA_real_, ave = NA_real_,
         factor_var = NA_real_)
})

# Combine train and test fit rows
cfa_fit_both <- bind_rows(cfa_fit_row, cfa_fit_test)
```

```
  list(cfa_tbl = cfa_tbl, cfa_params = cfa_params,
       cfa_fit = cfa_fit_both)
}
```

Component 2: Per-Fold IRT/GRM

```
fit_fold_irt <- function(train_df, test_df, cfg, survey, fold_id) {

  trust_items <- cfg$trust_items
  trust_train <- train_df |>
    select(all_of(trust_items)) |>
    mutate(across(everything(), as.numeric))
  trust_test <- test_df |>
    select(all_of(trust_items)) |>
    mutate(across(everything(), as.numeric))

  item_min <- min(unlist(trust_train), na.rm = TRUE)
  item_max <- max(unlist(trust_train), na.rm = TRUE)

  # Reverse-code for descending scales (LB)
  if (!cfg$trust_ascending) {
    irt_train <- trust_train |>
      mutate(across(everything(), ~ as.integer(item_max + 1L - .)))
    irt_test <- trust_test |>
      mutate(across(everything(), ~ as.integer(item_max + 1L - .)))
  } else {
    irt_train <- trust_train |>
      mutate(across(everything(), as.integer))
    irt_test <- trust_test |>
      mutate(across(everything(), as.integer))
  }

  irt_fit <- mirt(irt_train, 1, itemtype = "graded",
                  verbose = FALSE, SE = TRUE)

  # Discrimination + SE
  item_coefs <- coef(irt_fit, printSE = TRUE)
  irt_params <- map_dfr(trust_items, function(item) {
    mat <- item_coefs[[item]]
    tibble(survey = survey, fold_id = fold_id, item = item,
           a = round(mat["par", "a1"], 3),
           se_a = round(mat["SE", "a1"], 3))
  })
```

```r
# Q3 residual correlations
q3_mat <- residuals(irt_fit, type = "Q3", verbose = FALSE)
q3_pairs <- map_dfr(seq_along(trust_items), function(i) {
  map_dfr(seq_len(i - 1), function(j) {
    q3_val <- q3_mat[i, j]
    if (abs(q3_val) > Q3_THRESHOLD) {
      tibble(survey = survey, fold_id = fold_id,
             item1 = trust_items[j], item2 = trust_items[i],
             q3 = round(q3_val, 3))
    }
  })
})

irt_fit_row <- tibble(
  survey = survey, fold_id = fold_id,
  n_q3_notable = nrow(q3_pairs),
  max_q3 = round(if (nrow(q3_pairs) > 0) {
    max(abs(q3_pairs$q3))
  } else 0, 3)
)

# Theta estimates
theta_train_mat <- fscores(irt_fit, method = "EAP",
                           full.scores.SE = TRUE)
theta_train <- as.numeric(theta_train_mat[, 1])
se_train    <- as.numeric(theta_train_mat[, 2])

theta_test_mat <- fscores(
  irt_fit, response.pattern = as.matrix(irt_test),
  method = "EAP", full.scores.SE = TRUE)
theta_test <- as.numeric(theta_test_mat[, 1])
se_test    <- as.numeric(theta_test_mat[, 2])

ecdf_fn      <- ecdf(theta_train)
pctile_train <- round(ecdf_fn(theta_train) * 100)
pctile_test  <- round(ecdf_fn(theta_test)  * 100)
label_train  <- label_trust(theta_train, theta_train)
label_test   <- label_trust(theta_test,  theta_train)

irt_tbl <- bind_rows(
  tibble(survey = survey, fold_id = fold_id, split = "train",
         row_id = train_df$row_id, irt_theta = theta_train,
         irt_se = round(se_train, 3),
         irt_pctile = pctile_train, irt_label = label_train),
  tibble(survey = survey, fold_id = fold_id, split = "test",
         row_id = test_df$row_id, irt_theta = theta_test,
         irt_se = round(se_test, 3),
         irt_pctile = pctile_test, irt_label = label_test)
```

```
  )

  list(irt_tbl = irt_tbl, irt_params = irt_params,
       irt_fit = irt_fit_row, q3_pairs = q3_pairs,
       irt_model = irt_fit)
}
```

Component 3: Demographic Profiling

This component replaces MI, DIF, and theta regression with a data-driven approach that produces information LLMs can directly use. For each demographic variable, we compute:

1. **Screening:** Eta-squared with both IRT theta and CFA factor score. Demographics must pass the threshold (eta² 0.005) for at least one latent variable to be included in prompts for that survey. Both are reported so the reader can see whether CFA and IRT screening results diverge.
2. **Group means (Layer 1):** Mean trust rating per item per demographic group. Provides direction and magnitude.
3. **Model-based probabilities (Layer 2):** For each demographic group's mean theta, compute $P(X_j \geq k \mid \bar{\theta}_g)$ using the IRT model. This transforms psychometric parameters into probability statements the LLM can interpret.
4. **Conditional distributions (Layer 3):** Empirical $P(X_j = k \mid \text{group})$ for the strongest associations.

```
fit_fold_demo_profile <- function(train_df, irt_train_tbl,
                                  cfa_train_tbl, irt_model,
                                  cfg, survey, fold_id) {

  trust_items <- cfg$trust_items
  demo_vars   <- cfg$demo_vars
  factor_name <- cfg$factor_names
  fs_col      <- paste0("fs_", factor_name)

  item_min    <- min(as.numeric(unlist(
    train_df |> select(all_of(trust_items))
  )), na.rm = TRUE)
  item_max    <- max(as.numeric(unlist(
    train_df |> select(all_of(trust_items))
  )), na.rm = TRUE)
  n_cats      <- item_max - item_min + 1L

  # Merge both theta and CFA factor score onto training data
  merged <- train_df |>
    left_join(irt_train_tbl |> select(row_id, irt_theta),
              by = "row_id") |>
    left_join(cfa_train_tbl |> select(row_id, cfa_fs = !!sym(fs_col)),
```

```
            by = "row_id")

ecdf_theta <- ecdf(merged$irt_theta)

#   Step 1: Screening
# Eta-squared of each demographic with BOTH IRT theta and
# CFA factor score. A demographic passes if it meets the
# threshold for at least one latent variable.
screening <- map_dfr(demo_vars, function(dv) {
  if (!dv %in% names(merged)) {
    return(tibble(survey = survey, fold_id = fold_id,
                  demo_var = dv,
                  eta_sq_theta = NA_real_,
                  eta_sq_cfa = NA_real_,
                  F_theta = NA_real_, p_theta = NA_real_,
                  F_cfa = NA_real_, p_cfa = NA_real_,
                  n = 0L, passes = FALSE))
  }

  # IRT theta screening
  irt_result <- compute_eta_sq(merged$irt_theta, merged[[dv]])

  # CFA factor score screening
  cfa_result <- compute_eta_sq(merged$cfa_fs, merged[[dv]])

  tibble(
    survey       = survey,
    fold_id      = fold_id,
    demo_var     = dv,
    eta_sq_theta = irt_result$eta_sq,
    eta_sq_cfa   = cfa_result$eta_sq,
    F_theta      = irt_result$F_stat,
    p_theta      = irt_result$p_value,
    F_cfa        = cfa_result$F_stat,
    p_cfa        = cfa_result$p_value,
    n            = irt_result$n,
    # Passes if EITHER latent variable meets threshold
    passes       = (!is.na(irt_result$eta_sq) &
                      irt_result$eta_sq >= ETA_SQ_THRESHOLD) |
                   (!is.na(cfa_result$eta_sq) &
                      cfa_result$eta_sq >= ETA_SQ_THRESHOLD)
  )
})

# Demographics that pass screening for this fold
demo_passed <- screening |>
  filter(passes == TRUE) |>
  pull(demo_var)
```

```
cat(sprintf("    Demo screening: %s pass → [%s]\n",
            length(demo_passed),
            str_c(demo_passed, collapse = ", ")))

#   Step 2: Group means per item (Layer 1)
# Only for demographics that pass screening
group_means_items <- map_dfr(demo_passed, function(dv) {
  map_dfr(trust_items, function(item) {
    merged |>
      filter(!is.na(.data[[dv]]), !is.na(.data[[item]])) |>
      group_by(group = as.character(.data[[dv]])) |>
      summarise(
        mean_trust = round(mean(as.numeric(.data[[item]])), 3),
        sd_trust   = round(sd(as.numeric(.data[[item]])), 3),
        n          = n(),
        .groups    = "drop"
      ) |>
      mutate(survey = survey, fold_id = fold_id,
             demo_var = dv, item = item, .before = 1)
  })
})

#   Step 3: Model-based probabilities (Layer 2)
# For each demographic group, compute mean theta and then use
# the IRT model to get P(X_j >= k | mean_theta_group)
# This transforms IRT parameters into probability statements
group_means_theta <- map_dfr(demo_passed, function(dv) {
  merged |>
    filter(!is.na(.data[[dv]]), !is.na(irt_theta)) |>
    group_by(group = as.character(.data[[dv]])) |>
    summarise(
      mean_theta = mean(irt_theta),
      sd_theta   = sd(irt_theta),
      n          = n(),
      .groups    = "drop"
    ) |>
    mutate(
      pctile = round(ecdf_theta(mean_theta) * 100),
      survey = survey, fold_id = fold_id,
      demo_var = dv, .before = 1
    )
})

# Model-based conditional probabilities:
# P(X_j >= k | group mean theta) for each item × group
model_probs <- map_dfr(demo_passed, function(dv) {
```

```
  group_thetas <- group_means_theta |>
    filter(demo_var == dv) |>
    mutate(group = as.character(group)) |>
    select(group, mean_theta)

  pmap_dfr(group_thetas, function(group, mean_theta) {
    map_dfr(trust_items, function(item) {

      item_obj  <- extract.item(irt_model, item)
      raw_probs <- as.numeric(probtrace(item_obj, mean_theta))

      # Reverse probabilities for descending scales
      probs <- if (!cfg$trust_ascending) {
        rev(raw_probs)
      } else {
        raw_probs
      }

      # Category labels (original scale)
      cats <- seq(item_min, item_max)

      # Cumulative P(X >= k) for each category
      cum_probs <- map_dbl(seq_along(cats), function(k) {
        sum(probs[k:length(probs)])
      })

      # Find midpoint category
      midpoint <- ceiling(n_cats / 2) + item_min - 1L

      # P(higher trust) - scale-direction-aware
      # For ascending scales: higher trust = higher categories
      # For descending scales (after rev): higher trust = lower
      # category numbers on the original scale
      p_high_trust <- if (cfg$trust_ascending) {
        sum(probs[which(cats > midpoint)])
      } else {
        sum(probs[which(cats < midpoint)])
      }

      tibble(
        survey    = survey,
        fold_id   = fold_id,
        demo_var  = dv,
        group     = group,
        item      = item,
        modal_cat = cats[which.max(probs)],
        p_high_trust = round(p_high_trust, 3),
        p_at_modal  = round(max(probs), 3),
```

```
          prob_spread = list(
            tibble(category = cats, prob = round(probs, 4),
                   p_geq = round(cum_probs, 4))
          )
        )
      })
    })
  })

  #   Step 4: Conditional distributions (Layer 3)
  # Empirical P(X_j = k | group) for all items × passed demos
  conditional_dists <- map_dfr(demo_passed, function(dv) {
    map_dfr(trust_items, function(item) {
      merged |>
        filter(!is.na(.data[[dv]]),
               !is.na(.data[[item]])) |>
        count(group = as.character(.data[[dv]]),
              category = as.integer(.data[[item]])) |>
        group_by(group) |>
        mutate(prop = round(n / sum(n), 4)) |>
        ungroup() |>
        mutate(survey = survey, fold_id = fold_id,
               demo_var = dv, item = item, .before = 1)
    })
  })

  list(
    screening         = screening,
    group_means_items = group_means_items,
    group_means_theta = group_means_theta,
    model_probs       = model_probs,
    conditional_dists = conditional_dists
  )
}
```

Main Fitting Loop

```
fit_all_folds <- function(fold_df, cfg, survey_name) {

  fold_ids <- sort(unique(fold_df$fold_id))

  results <- map(fold_ids, function(fi) {

    cat("\n== Fitting:", survey_name, "| Fold", fi, "==\n")
```

```
  train_df <- fold_df |>
    filter(fold_id == fi, split == "train") |>
    arrange(row_id)
  test_df <- fold_df |>
    filter(fold_id == fi, split == "test") |>
    arrange(row_id)

  cat("  CFA...\n")
  cfa_out <- fit_fold_cfa(train_df, test_df, cfg,
                          survey_name, fi)

  cat("  IRT/GRM...\n")
  irt_out <- fit_fold_irt(train_df, test_df, cfg,
                          survey_name, fi)

  cat("  Demographic profiling...\n")
  irt_train <- irt_out$irt_tbl |> filter(split == "train")
  cfa_train <- cfa_out$cfa_tbl |> filter(split == "train")
  demo_out <- fit_fold_demo_profile(
    train_df, irt_train, cfa_train, irt_out$irt_model,
    cfg, survey_name, fi)

  cat("  Done.\n")

  list(cfa = cfa_out, irt = irt_out, demo = demo_out)
})

#   Assemble all outputs
list(
  cfa_tbl      = map_dfr(results, function(x) x$cfa$cfa_tbl),
  cfa_params   = map_dfr(results, function(x) x$cfa$cfa_params),
  cfa_fit      = map_dfr(results, function(x) x$cfa$cfa_fit),
  irt_tbl      = map_dfr(results, function(x) x$irt$irt_tbl),
  irt_params   = map_dfr(results, function(x) x$irt$irt_params),
  irt_fit      = map_dfr(results, function(x) x$irt$irt_fit),
  q3_pairs     = map_dfr(results, function(x) x$irt$q3_pairs),
  irt_models   = map(results, function(x) x$irt$irt_model) |>
                   set_names(fold_ids),
  # Demographic profiling
  demo_screening    = map_dfr(results,
                              function(x) x$demo$screening),
  group_means_items = map_dfr(results,
                              function(x) x$demo$group_means_items),
  group_means_theta = map_dfr(results,
                              function(x) x$demo$group_means_theta),
  model_probs       = map_dfr(results,
                              function(x) x$demo$model_probs),
  conditional_dists = map_dfr(results,
```

```
                                function(x) x$demo$conditional_dists)
  )
}

lapop_results <- fit_all_folds(folds_list$lapop, config$lapop,
                               "lapop")
lb_results    <- fit_all_folds(folds_list$lb, config$lb, "lb")
```

Assemble Output Objects

```
# CFA
cfa_list       <- list(lapop = lapop_results$cfa_tbl,
                       lb = lb_results$cfa_tbl)
cfa_params_tbl <- bind_rows(lapop_results$cfa_params,
                            lb_results$cfa_params)
cfa_fit_tbl    <- bind_rows(lapop_results$cfa_fit,
                            lb_results$cfa_fit)

# IRT
irt_list       <- list(lapop = lapop_results$irt_tbl,
                       lb = lb_results$irt_tbl)
irt_params_tbl <- bind_rows(lapop_results$irt_params,
                            lb_results$irt_params)
irt_fit_tbl    <- bind_rows(lapop_results$irt_fit,
                            lb_results$irt_fit)
q3_pairs_tbl   <- bind_rows(lapop_results$q3_pairs,
                            lb_results$q3_pairs)
irt_models     <- list(lapop = lapop_results$irt_models,
                       lb = lb_results$irt_models)

# Demographic profiling
demo_screening_tbl    <- bind_rows(lapop_results$demo_screening,
                                   lb_results$demo_screening)
group_means_items_tbl <- bind_rows(
  lapop_results$group_means_items,
  lb_results$group_means_items)
group_means_theta_tbl <- bind_rows(
  lapop_results$group_means_theta,
  lb_results$group_means_theta)
model_probs_tbl       <- bind_rows(lapop_results$model_probs,
                                   lb_results$model_probs)
conditional_dists_tbl <- bind_rows(
  lapop_results$conditional_dists,
  lb_results$conditional_dists)
```

```
# Assemble the demo_profile_tbl as a list of the three layers
# plus model probabilities
demo_profile_tbl <- list(
  screening         = demo_screening_tbl,
  group_means_items = group_means_items_tbl,
  group_means_theta = group_means_theta_tbl,
  model_probs       = model_probs_tbl,
  conditional_dists = conditional_dists_tbl
)

cat("Assembly complete.\n")
```

```
Assembly complete.
```

Validation

```
bind_rows(
  tibble(Survey = "lapop",
         CFA_rows = nrow(cfa_list$lapop),
         IRT_rows = nrow(irt_list$lapop),
         Fold_rows = nrow(folds_list$lapop),
         CFA_ok = nrow(cfa_list$lapop) == nrow(folds_list$lapop),
         IRT_ok = nrow(irt_list$lapop) == nrow(folds_list$lapop)),
  tibble(Survey = "lb",
         CFA_rows = nrow(cfa_list$lb),
         IRT_rows = nrow(irt_list$lb),
         Fold_rows = nrow(folds_list$lb),
         CFA_ok = nrow(cfa_list$lb) == nrow(folds_list$lb),
         IRT_ok = nrow(irt_list$lb) == nrow(folds_list$lb))
) |> knitr::kable(align = "lccccl")
```

Survey	CFA_rows	IRT_rows	Fold_rows	CFA_ok	IRT_ok
lapop	7150	7150	7150	TRUE	TRUE
lb	5365	5365	5365	TRUE	TRUE

CFA Fit Summary

```
cfa_fit_tbl |>
  group_by(survey, split) |>
  summarise(across(c(cfi, tli, rmsea, srmr, omega, ave),
```

```
                    mean, na.rm = TRUE), .groups = "drop") |>
  mutate(across(where(is.numeric), function(x) round(x, 3))) |>
  knitr::kable(col.names = c("Survey", "Split", "CFI", "TLI",
                             "RMSEA", "SRMR", " ", "AVE"),
               align = "llccccccc")
```

Table 2: Mean CFA Fit Across 5 Folds (Train = model estimation, Test = H2 baseline)

Survey	Split	CFI	TLI	RMSEA	SRMR		AVE
lapop	test	0.945	0.930	0.086	0.038	0.920	0.522
lapop	train	0.957	0.945	0.076	0.030	0.921	0.521
lb	test	0.980	0.965	0.069	0.028	0.777	0.437
lb	train	0.984	0.969	0.069	0.023	0.783	0.437

IRT Discrimination Summary

```
irt_params_tbl |>
  group_by(survey, item) |>
  summarise(mean_a = round(mean(a), 3),
            sd_a = round(sd(a), 3), .groups = "drop") |>
  arrange(survey, desc(mean_a)) |>
  knitr::kable(col.names = c("Survey", "Item", "Mean a",
                             "SD(a)"),
               align = "llcc")
```

Table 3: Mean Item Discrimination Across 5 Folds

Survey	Item	Mean a	SD(a)
lapop	justice_system	3.304	0.052
lapop	auditor	2.968	0.075
lapop	public_ministry	2.792	0.058
lapop	supreme_court	2.552	0.122
lapop	legislature	2.263	0.039
lapop	elections	2.210	0.057
lapop	political_parties	2.134	0.047
lapop	electoral_tribunal	1.995	0.068
lapop	municipality	1.985	0.069
lapop	police	1.918	0.034
lapop	media	1.368	0.074
lapop	armed_forces	1.333	0.034
lb	Natl_Govt	2.668	0.199
lb	Judiciary	2.139	0.085

Table 3: Mean Item Discrimination Across 5 Folds

Survey	Item	Mean a	SD(a)
lb	Parties	2.124	0.080
lb	Electoral_Inst	2.037	0.083
lb	President	2.010	0.161
lb	Congress	1.994	0.074

Demographic Screening Results

```
demo_screening_tbl |>
  group_by(survey, demo_var) |>
  summarise(
    mean_eta_theta = round(mean(eta_sq_theta, na.rm = TRUE), 4),
    mean_eta_cfa   = round(mean(eta_sq_cfa, na.rm = TRUE), 4),
    n_folds_pass   = sum(passes),
    .groups = "drop"
  ) |>
  mutate(
    max_eta = pmax(mean_eta_theta, mean_eta_cfa, na.rm = TRUE),
    verdict = case_when(
      max_eta >= 0.01 ~ "KEEP",
      max_eta >= ETA_SQ_THRESHOLD ~ "MARGINAL-KEEP",
      TRUE ~ "DROP"
    )
  ) |>
  arrange(survey, desc(max_eta)) |>
  select(-max_eta) |>
  knitr::kable(
    col.names = c("Survey", "Demographic", " ²( )",
                  " ²(CFA)", "Folds Pass", "Verdict"),
    align = "llcccc"
  )
```

Table 4: Demographic Screening: Eta² with IRT Theta and CFA Factor Score (Averaged Across 5 Folds)

Survey	Demographic	²( )	²(CFA)	Folds Pass	Verdict
lapop	Edu	0.0209	0.0201	5	KEEP
lapop	age_cat	0.0122	0.0119	5	KEEP
lapop	Employment	0.0088	0.0081	4	MARGINAL-KEEP
lapop	Urban	0.0055	0.0067	4	MARGINAL-KEEP
lapop	male	0.0005	0.0007	0	DROP

Table 4: Demographic Screening: Eta² with IRT Theta and CFA Factor Score (Averaged Across 5 Folds)

Survey	Demographic	^{2}( )	^{2}(CFA)	Folds Pass	Verdict
lb	age_cat	0.0137	0.0189	5	KEEP
lb	Edu	0.0152	0.0129	5	KEEP
lb	Employment	0.0110	0.0116	5	KEEP
lb	male	0.0032	0.0024	0	DROP
lb	Urban	0.0004	0.0003	0	DROP

Demographic Group Means (Theta)

```
group_means_theta_tbl |>
  group_by(survey, demo_var, group) |>
  summarise(
    mean_theta = round(mean(mean_theta), 3),
    mean_pctile = round(mean(pctile)),
    mean_n = round(mean(n)),
    .groups = "drop"
  ) |>
  arrange(survey, demo_var, desc(mean_theta)) |>
  knitr::kable(
    col.names = c("Survey", "Demographic", "Group",
                  "Mean ", "Percentile", "N"),
    align = "lllccc"
  )
```

Table 5: Mean Theta by Demographic Group (Averaged Across 5 Folds, Surviving Demographics Only)

Survey	Demographic	Group	Mean	Percentile	N
lapop	Edu	none	0.345	64	20
lapop	Edu	primary	0.223	60	218
lapop	Edu	secondary	-0.014	50	674
lapop	Edu	higher_ed	-0.195	41	232
lapop	Employment	Retired	0.188	59	69
lapop	Employment	house_wife	0.107	55	303
lapop	Employment	Student	0.083	54	70
lapop	Employment	Employed	-0.065	46	614
lapop	Employment	Unemployed	-0.106	44	88
lapop	Urban	rural	0.156	58	223
lapop	Urban	urban	-0.036	48	921
lapop	age_cat	60+	0.236	61	186

Table 5: Mean Theta by Demographic Group (Averaged Across 5 Folds, Surviving Demographics Only)

Survey	Demographic	Group	Mean	Percentile	N
lapop	age_cat	18-29	-0.027	49	374
lapop	age_cat	30-44	-0.043	48	289
lapop	age_cat	45-59	-0.068	47	295
lb	Edu	higher_ed	0.222	55	158
lb	Edu	none	0.027	49	19
lb	Edu	secondary	-0.037	45	559
lb	Edu	primary	-0.118	41	122
lb	Employment	Student	0.244	58	69
lb	Employment	Retired	0.049	48	25
lb	Employment	Employed	0.016	47	449
lb	Employment	house_wife	-0.042	45	210
lb	Employment	Unemployed	-0.155	40	106
lb	age_cat	18-29	0.146	53	271
lb	age_cat	60+	0.020	47	104
lb	age_cat	45-59	-0.066	44	198
lb	age_cat	30-44	-0.100	42	285

Model-Based Probability Context Example

```
# Show the top 5 items by between-group probability range
# for each survey (fold 1 only for display)
model_probs_tbl |>
  filter(fold_id == 1) |>
  group_by(survey, demo_var, item) |>
  summarise(
    range_p = max(p_high_trust) - min(p_high_trust),
    .groups = "drop"
  ) |>
  group_by(survey) |>
  slice_max(range_p, n = 6) |>
  ungroup() |>
  left_join(
    model_probs_tbl |>
      filter(fold_id == 1) |>
      select(survey, demo_var, item, group,
             p_high_trust, modal_cat),
    by = c("survey", "demo_var", "item")
  ) |>
  select(survey, demo_var, item, group,
         p_high_trust, modal_cat) |>
  arrange(survey, demo_var, item,
```

```
        desc(p_high_trust)) |>
knitr::kable(
  col.names = c("Survey", "Demo", "Item", "Group",
                "P(high trust)", "Modal Cat"),
  align = "lllccc"
)
```

Table 6: IRT Model-Based P(high trust) by Demographic Group — Fold 1, Top Items

Survey	Demo	Item	Group	P(high trust)	Modal Cat
lapop	Edu	auditor	none	0.608	5
lapop	Edu	auditor	primary	0.552	5
lapop	Edu	auditor	secondary	0.393	4
lapop	Edu	auditor	higher_ed	0.276	4
lapop	Edu	justice_system	none	0.594	5
lapop	Edu	justice_system	primary	0.531	5
lapop	Edu	justice_system	secondary	0.356	4
lapop	Edu	justice_system	higher_ed	0.233	4
lapop	Edu	legislature	none	0.668	5
lapop	Edu	legislature	primary	0.628	5
lapop	Edu	legislature	secondary	0.506	5
lapop	Edu	legislature	higher_ed	0.404	4
lapop	Edu	public_ministry	none	0.708	5
lapop	Edu	public_ministry	primary	0.661	5
lapop	Edu	public_ministry	secondary	0.512	5
lapop	Edu	public_ministry	higher_ed	0.386	4
lapop	Edu	supreme_court	none	0.724	5
lapop	Edu	supreme_court	primary	0.680	5
lapop	Edu	supreme_court	secondary	0.539	5
lapop	Edu	supreme_court	higher_ed	0.417	4
lapop	Employment	justice_system	Retired	0.568	5
lapop	Employment	justice_system	house_wife	0.444	4
lapop	Employment	justice_system	Student	0.349	4
lapop	Employment	justice_system	Employed	0.317	4
lapop	Employment	justice_system	Unemployed	0.280	4
lb	Edu	Natl_Govt	higher_ed	0.016	3
lb	Edu	Natl_Govt	none	0.013	3
lb	Edu	Natl_Govt	secondary	0.007	3
lb	Edu	Natl_Govt	primary	0.006	3
lb	Edu	President	higher_ed	0.060	3
lb	Edu	President	none	0.050	3
lb	Edu	President	secondary	0.034	3
lb	Edu	President	primary	0.027	3
lb	Employment	Judiciary	Student	0.018	3
lb	Employment	Judiciary	Employed	0.011	3
lb	Employment	Judiciary	Retired	0.011	3

Table 6: IRT Model-Based P(high trust) by Demographic Group — Fold 1, Top Items

Survey	Demo	Item	Group	P(high trust)	Modal Cat
lb	Employment	Judiciary	house_wife	0.009	3
lb	Employment	Judiciary	Unemployed	0.007	3
lb	Employment	Natl_Govt	Student	0.016	3
lb	Employment	Natl_Govt	Employed	0.009	3
lb	Employment	Natl_Govt	Retired	0.009	3
lb	Employment	Natl_Govt	house_wife	0.007	3
lb	Employment	Natl_Govt	Unemployed	0.005	3
lb	Employment	President	Student	0.061	3
lb	Employment	President	Retired	0.040	3
lb	Employment	President	Employed	0.039	3
lb	Employment	President	house_wife	0.033	3
lb	Employment	President	Unemployed	0.025	3
lb	age_cat	President	18-29	0.050	3
lb	age_cat	President	45-59	0.035	3
lb	age_cat	President	60+	0.034	3
lb	age_cat	President	30-44	0.030	3

Contour Plots: Theta Distribution by Demographic Group

```
# Build combined data for ridgeline plots
# Use fold 1 training data for display
ridge_data <- map_dfr(c("lapop", "lb"), function(survey_name) {

  # Get surviving demographics for this survey
  # A demographic survives if max(eta_theta, eta_cfa) >= threshold
  survivors <- demo_screening_tbl |>
    filter(survey == survey_name) |>
    group_by(demo_var) |>
    summarise(
      mean_eta_theta = mean(eta_sq_theta, na.rm = TRUE),
      mean_eta_cfa   = mean(eta_sq_cfa, na.rm = TRUE),
      .groups = "drop"
    ) |>
    mutate(max_eta = pmax(mean_eta_theta, mean_eta_cfa,
                          na.rm = TRUE)) |>
    filter(max_eta >= ETA_SQ_THRESHOLD) |>
    pull(demo_var)

  # Get fold 1 training data with theta
  train_df <- folds_list[[survey_name]] |>
    filter(fold_id == 1, split == "train")

  theta_df <- irt_list[[survey_name]] |>
    filter(fold_id == 1, split == "train") |>
    select(row_id, irt_theta)

  merged <- train_df |>
    left_join(theta_df, by = "row_id")

  # Pivot demographics to long format
  map_dfr(survivors, function(dv) {
    merged |>
      filter(!is.na(.data[[dv]]), !is.na(irt_theta)) |>
      transmute(
        survey = if_else(survey_name == "lapop",
                         "LAPOP Mexico (7-point)",
                         "Latinobarómetro Colombia (4-point)"),
        demo_var = dv,
        group = as.character(.data[[dv]]),
        irt_theta = irt_theta
      )
  })
})
```

```r
# Ridgeline plot
ggplot(ridge_data,
       aes(x = irt_theta, y = group, fill = group)) +
  geom_density_ridges(
    alpha = 0.7, scale = 1.2,
    rel_min_height = 0.01
  ) +
  facet_grid(demo_var ~ survey, scales = "free_y",
             space = "free_y") +
  scale_fill_viridis_d(option = "D", end = 0.85) +
  labs(
    x = expression(IRT ~ theta ~ (latent ~ trust)),
    y = NULL
  ) +
  theme_minimal(base_size = 11) +
  theme(
    legend.position = "none",
    strip.text.y = element_text(angle = 0, face = "bold"),
    strip.text.x = element_text(face = "bold")
  )
```

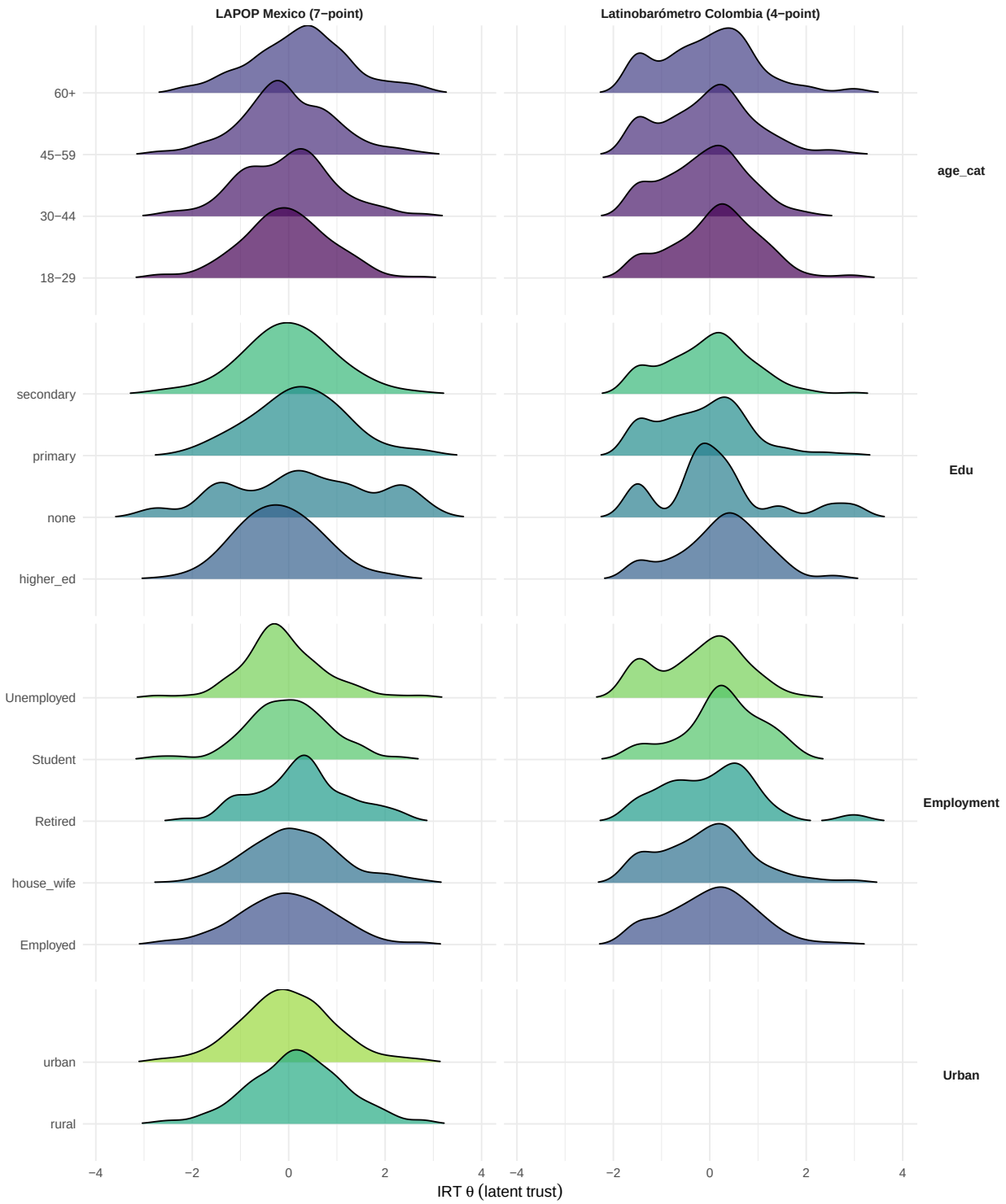

Figure 1: IRT Theta Distribution by Demographic Group. Only demographics that passed screening are shown. Width of ridges reflects the density of respondents at each theta level.

Generate Masking Table

Each test respondent is randomly assigned 1–4 missing items, with the specific masked items selected at random.

```
masking_tbl <- map_dfr(names(config), function(survey_name) {
  items <- config[[survey_name]]$trust_items

  map_dfr(1:N_FOLDS, function(fold_i) {
    test_ids <- folds_list[[survey_name]] |>
      filter(fold_id == fold_i, split == "test") |>
      pull(row_id)

    n_test   <- length(test_ids)
    n_masked <- sample(1:4, n_test, replace = TRUE)

    map2_dfr(test_ids, n_masked, function(rid, k) {
      tibble(
        survey      = survey_name,
        fold_id     = fold_i,
        row_id      = rid,
        n_masked    = k,
        masked_item = list(sample(items, size = k))
      )
    })
  })
})

cat("Masking table:", nrow(masking_tbl), "rows\n\n")
```

```
Masking table: 2503 rows
```

```
masking_tbl |>
  count(survey, n_masked) |>
  knitr::kable(caption = "Masking Distribution",
               align = "lcc")
```

Table 7: Masking Distribution

survey	n_masked	n
lapop	1	336
lapop	2	358
lapop	3	377
lapop	4	359
lb	1	272

survey	n_masked	n
lb	2	250
lb	3	283
lb	4	268

Save Outputs

```
# Primary fold data
output_path <- file.path(results_dir, "fold_data.RData")
save(folds_list, cfa_list, irt_list, masking_tbl,
     file = output_path)
cat("Saved fold_data.RData:",
    round(file.size(output_path) / 1e6, 2), "MB\n")
```

```
Saved fold_data.RData: 0.51 MB
```

```
# Also save masking table as standalone RDS
saveRDS(masking_tbl, file.path(results_dir,
                               "masking_table.rds"))
cat("Saved masking_table.rds\n")
```

```
Saved masking_table.rds
```

```
# Psychometric reporting tables
reporting_path <- file.path(results_dir,
                            "cfa_reporting.RData")
save(cfa_params_tbl, cfa_fit_tbl,
     irt_params_tbl, irt_fit_tbl, q3_pairs_tbl,
     demo_profile_tbl,
     file = reporting_path)
cat("Saved cfa_reporting.RData:",
    round(file.size(reporting_path) / 1e6, 2), "MB\n")
```

```
Saved cfa_reporting.RData: 0.12 MB
```

```
# IRT model objects (needed for imputation scripts)
irt_path <- file.path(results_dir, "irt_models.RData")
save(irt_models, file = irt_path)
cat("Saved irt_models.RData:",
    round(file.size(irt_path) / 1e6, 2), "MB\n")
```

```
Saved irt_models.RData: 0.72 MB
```

```
# Summary of saved objects
tibble(
  Object = c("folds_list$lapop", "folds_list$lb",
             "cfa_list$lapop", "cfa_list$lb",
             "irt_list$lapop", "irt_list$lb",
             "cfa_params_tbl", "cfa_fit_tbl",
             "irt_params_tbl", "irt_fit_tbl",
             "q3_pairs_tbl",
             "demo_profile_tbl$screening",
             "demo_profile_tbl$group_means_items",
             "demo_profile_tbl$group_means_theta",
             "demo_profile_tbl$model_probs",
             "demo_profile_tbl$conditional_dists",
             "masking_tbl",
             "irt_models$lapop", "irt_models$lb"),
  Rows = c(nrow(folds_list$lapop), nrow(folds_list$lb),
           nrow(cfa_list$lapop), nrow(cfa_list$lb),
           nrow(irt_list$lapop), nrow(irt_list$lb),
           nrow(cfa_params_tbl), nrow(cfa_fit_tbl),
           nrow(irt_params_tbl), nrow(irt_fit_tbl),
           nrow(q3_pairs_tbl),
           nrow(demo_profile_tbl$screening),
           nrow(demo_profile_tbl$group_means_items),
           nrow(demo_profile_tbl$group_means_theta),
           nrow(demo_profile_tbl$model_probs),
           nrow(demo_profile_tbl$conditional_dists),
           nrow(masking_tbl),
           length(irt_models$lapop),
           length(irt_models$lb)),
  Saved_In = c(rep("fold_data", 6),
               rep("cfa_reporting", 10),
               "fold_data",
               rep("irt_models", 2))
) |> knitr::kable(align = "lcc")
```

Object	Rows	Saved_In
folds_list$lapop$|7150|fold_data||folds_list$lb	5365	fold_data
cfa_list$lapop$|7150|fold_data||cfa_list$lb	5365	fold_data
irt_list$lapop$|7150|fold_data||irt_list$lb	5365	fold_data
cfa_params_tbl	190	cfa_reporting
cfa_fit_tbl	20	cfa_reporting
irt_params_tbl	90	cfa_reporting
irt_fit_tbl	10	cfa_reporting
q3_pairs_tbl	225	cfa_reporting

Object	Rows	Saved_In
demo_profile_tbl*screening*|50|$cfa_reporting$||$demo_profile_blgroup$	1206	means_items
demo_profile_tbl*group_means_theta*|133|$cfa_reporting$||$demo_profile_blmodel$	1206	model_probs
demo_profile_tbl*conditional_dists*|7206|$cfa_reporting$||$masking_bl$|2503|$fold_data$	5	irt_models
irt_models$lb	5	irt_models